

RadFor-360 — Auditoría de Arquitectura y Plan de Optimización (Economía · Seguridad · Escalabilidad)

Alcance: raíz `c:\2026 AI EGIOC5\Antigravity JS` (`package.json` , `name: "brevo-total-integration"` — nombre real de producto "RadFor-360", ver `RadarApp.jsx:128` y `docs/RADIOGRAFIA_TECNICA_RADFOR360.md`). No incluye `proyectos/Proy_03_RadarFondos/` (sin integración real confirmada con la raíz).

Fecha: 2026-08-06. **Método:** auditoría de solo lectura ejecutada por el subagente `architect` (Read/Grep/Glob, sin permisos de escritura), sobre el criterio explícito del usuario: **economía primero en esta etapa, más seguridad de nivel SaaS enterprise, más escalabilidad a futuro sin sobreconstruir hoy**. Hallazgos clave re-verificados de forma independiente antes de este documento (ver nota al final de cada sección marcada "  verificado").

Documentos previos con los que este informe se relaciona, sin repetirlos:

- `docs/RADIOGRAFIA_TECNICA_RADFOR360.md` — inventario forense de MVP/stubs/RBAC/FinOps (2026-08-05). Este informe construye encima, corrige un punto (ver §0) y añade la capa prescriptiva de optimización que aquél no tenía como objetivo.
- `docs/ARQUITECTURA_AGENTICA_ANTIGRAVITY.md` — auditoría del ecosistema de agentes/skills (no de RadFor-360 en sí).

Cambios ya aplicados en la sesión de trabajo previa a esta auditoría (verificados por el arquitecto y confirmados aquí): `bypass X-Local-Dev` eliminado de `FirebaseAuthMiddleware.js`; `session-manager.js` migrado de `Map` local a `cache.js` (Upstash Redis-o-memoria); WS de radar marcando `_simulado: true`; `AGENTS.md` corregido a Claude/Anthropic; custom claim `role: admin` asignado a la cuenta real del dueño en Firebase.

0. Hallazgo transversal que reencuadra todo lo demás — el SPA no tiene login funcional

 **Verificado de forma independiente, código completo leído.**

- `public/src/pages/InicioPage.jsx` (69 líneas): maqueta puramente visual. No importa `useState`. Los `<input type="email">` / `<input type="password">` (líneas 32-36, 46-50) no tienen `onChange`. El botón "Ingresar" (línea 54-57) no tiene `onClick`. No hace nada al hacer clic.
- `public/src/main.jsx` (13 líneas, completo): monta `<App/>` dentro de `<BrowserRouter>` sin ningún `AuthProvider`, contexto de sesión, ni wrapper de Firebase.
- Búsqueda de `signInWith|firebase/auth|session/login|getIdToken` en todo `public/src/` → **0 coincidencias**.
- El único flujo de login que **sí funciona** en el repo es `public/test_auth.html` — página HTML aislada, no enlazada desde ninguna ruta de `App.jsx`, que hace `signInWithPopup` con Google y solo imprime el resultado en consola; no llama a `/api/session/login`, no persiste el token, no navega a ningún sitio.

Consecuencia: todos los endpoints protegidos (`/api/formulador/*`, `/api/radar/*`, `/api/chat`, `/api/minimax/chat`, `/api/session/*` salvo `login/verify`, comunicaciones, settings, trello, github) solo son alcanzables por llamada directa a la API con un token obtenido a mano — nunca desde la aplicación web desplegada tal como existe hoy. Esto reencuadra tanto el análisis de costo (§2: el consumo real de Claude/Tavily es ~cero porque el producto es inalcanzable, no porque haya buena política de FinOps) como el de seguridad (§3: la superficie de ataque real es baja porque no hay puerta de entrada, no porque las mitigaciones ya construidas estén todas probadas en producción con tráfico real).

1. Mapa de arquitectura real actual (post-cambios de la sesión previa)

Capa	Estado verificado	Evidencia
Auth Firebase (backend)	Gate universal sin excepciones; bypass eliminado	<code>FirebaseAuthMiddleware.js</code> (17 líneas, sin condicional de entorno)
Auth Firebase (frontend)	No existe — ver §0	<code>InicioPage.jsx</code> , <code>main.jsx</code>
JWT propio (<code>session-manager.js</code>)	Backend completo y correcto; frontend nunca lo invoca	<code>session-manager.js</code> ; 0 referencias en <code>public/src/</code>
Sesiones (<code>activeSessions</code>)	Ya no es un <code>Map</code> puro — persiste vía <code>cacheGet/cacheSet/cacheDel</code>	<code>session-manager.js</code>
Cache (<code>cache.js</code>)	Upstash Redis REST primario + fallback en memoria — hoy cae a memoria en la práctica (<code>UPSTASH_REDIS_REST_URL/TOKEN</code> vacíos en <code>.env</code>)	<code>cache.js</code>
<code>radarData</code>	Sigue en memoria, array simple, reinicio = vuelve al seed	<code>server.js:59</code>
<code>activeQueries</code> (anti-flood)	Deliberadamente por-proceso, diseño correcto documentado	<code>session-manager.js</code>
Pipeline IA Radar (<code>m1Pipeline.js</code>)	Real, Claude+Tavily con tool-use, cache dual 24h, sin consumidor UI	<code>m1Pipeline.js</code> ; <code>RadarApp.jsx</code> solo llama <code>/api/convocatorias</code>
WS simulado	Marca <code>_simulado: true</code> , sigue desconectado de <code>m1Pipeline.js</code>	<code>server.js:386-407</code>
Formulador (RLS+COP)	Backend + esquema SQL reales; cero UI React lo consume	<code>FormuladorRouter.js</code> , migraciones <code>001-005</code>
<code>pgClient.js</code>	Código muerto confirmado — 0 imports en <code>src/</code> , <code>DATABASE_URL</code> vacío	✅ verificado, <code>grep</code> sin resultados
Backends paralelos (<code>backend_fastapi.py</code> , <code>server-sim.js</code>)	Sin camino a producción; FastAPI tiene <code>allow_origins=["*"]</code>	<code>render.yaml</code> no los referencia
Deploy real	Un solo proceso Node, Render <code>plan: free</code> , sin réplicas	<code>render.yaml:1-8</code>

2. Análisis de costo recurrente por componente

Componente	Uso real hoy	Disparador	Riesgo/desperdicio
Anthropic Claude	~Cero tráfico real (nadie puede loguearse)	Por request	Sin ningún límite por usuario/día — si el login se arregla sin ponerle tope, un usuario puede disparar llamadas Claude+Tavily sin límite
Tavily	Igual que Claude, mismo disparador	Dentro del loop de Claude (hasta 3/pipeline)	Cache key es MD5 del JSON literal, sin normalizar (<code>"Vivienda rural"</code> ≠ <code>"vivienda rural "</code> para el cache) — misses evitables
OpenRouter	Cero — sin consumidor en frontend	Nada lo llama	Puro standby; comentario <code>server.js:279</code> ("acceso público") está desactualizado —

Componente	Uso real hoy	Disparador	Riesgo/desperdicio
			el gate Sí aplica, el código es más estricto que el comentario
Upstash Redis	Configurado en código, vacío en .env → 100% cae a memoria	—	Pieza más barata de arreglar de todo el informe: tier gratuito de Upstash sobra para el tráfico actual
Supabase	Ping en cada deploy + /api/health , con SERVICE_KEY	Por deploy	Riesgo de plataforma (pausa por inactividad en free tier) ya mitigado por diseño (db-check.js aborta el deploy si falla)
Firebase Auth	Sin costo relevante en tier gratuito	—	No es driver de costo
Firestore	AuditLogger + generar_reporte.cjs escribiendo cada 10 min sin condición	Timer fijo	144 escrituras/día garantizadas solo para mantener un dashboard de conteo de carpetas — insignificante hoy, pero es el mismo patrón "costo por timer" que el WS simulado, a vigilar si se agregan más timers
Render	plan: free , un proceso	Fijo	Gratis hoy; su ciclo de suspensión/reactivación resetea radarData y (mientras Upstash siga vacío) las sesiones, sin mediar deploy
GitHub/Trello/Brevo/Stitch/Mongo/Groq/Gemini	Presentes en .env , sin driver de costo activo salvo uso manual de GitHub/Trello	Manual	Groq/Gemini confirmado sin ningún código que los invoque

La pregunta cara: ¿conectar el WS "live" a IA real?

Recomendación bajo criterio de economía, en orden de preferencia: 1. **Búsqueda on-demand** disparada por el usuario (botón "Actualizar radar") — costo proporcional a uso real. 2. Si se quiere sensación de "vivo" sin costo por usuario: **un cron de baja frecuencia único para todo el sistema** (cada 6-12h, no por cliente conectado) que alimente radarData desde m1Pipeline.js y lo transmita a todos los WS ya conectados — un pipeline sirve a N usuarios. 3. Reutilizar el cache dual de 24h que ya existe (cache.js , m1Pipeline.js) también para el cron, no solo para /search . 4. Añadir un límite diario simple por uid antes de exponer /api/radar/search más ampliamente — hoy no existe ninguno. 5. **Qué NO hacer todavía:** scheduler distribuido, colas (INNGEST_EVENT_KEY está vacío y sin código que lo use), ni un servicio separado. Un setInterval de baja frecuencia dentro del mismo server.js (real en vez de simulado) alcanza para el tráfico actual.

3. Auditoría de seguridad específica de RadFor-360

3.1 Firebase Auth vs. JWT propio — sin ambigüedad real, pero con un eslabón roto

El gate universal exige siempre Firebase; el JWT propio requiere Firebase para emitirse y hoy **nadie lo invoca desde el frontend** (§0). El custom claim role: admin recién asignado sí llega a req.user vía Firebase, pero **ningún endpoint de negocio lee req.user.role** — solo revokeSession lee el rol, y ese rol viene del JWT propio, no de Firebase. El

RBAC de admin sigue siendo inalcanzable desde cualquier flujo real de usuario porque el eslabón que lo activaría (`/api/session/login` desde el frontend) no existe.

3.2 Validación de entrada — patrón dominante en todo el árbol

Sin librería de esquemas (`zod` / `joi` / `express-validator`) en ningún punto de RadFor-360. Patrón consistente: check manual de 1-2 campos obligatorios. Los más débiles: `POST /api/settings/trello` (ninguna validación, `req.body` completo pasa directo), `POST /api/execute` (no-op, sin validación porque no ejecuta nada). Los más fuertes: `/api/formulador/*` (valida formato UUID de tenant e id vía regex).

3.3 Manejo de secretos

`.env` contiene valores reales (no plantilla) para 13 credenciales de servicios de pago/producción. Mitigante confirmado: `.gitignore` excluye `.env` y `config/serviceAccountKey.json` explícitamente (doble entrada). **Pendiente de verificar** (fuera del alcance de un agente sin `Bash /git`): si `.env` estuvo trackeado en algún commit histórico antes de esta regla — recomendado correr `git log --all --full-history -- .env` antes de dar el tema por cerrado.

Hallazgo menor: `FirebaseAdmin.js:22` loguea `""Auth middleware operará en modo bypass local""` si falla la carga del service account — **mensaje obsoleto**, ya no existe ningún bypass; el comportamiento real hoy sería fail-closed (401 en todo). El log podría hacer creer a un operador que hay un agujero activo cuando el sistema en realidad falla cerrado.

3.4 CORS

`ALLOWED_ORIGINS` por defecto solo `localhost`; en producción depende de variable manual en Render (ya documentado con advertencia en el propio `render.yaml`). `DEV_HEADERS / X-Local-Dev` en la lista de headers permitidos es ahora un vestigio inofensivo sin consumidor, tras eliminar el bypass.

3.5 Rate limiting

No existe ninguno — ni por IP, ni por usuario, ni por endpoint, en ningún punto de RadFor-360.

`acquireQuery / releaseQuery` no es rate limiting: solo evita que el mismo `uid` dispare la misma query duplicada en simultáneo.

3.6 Brecha entre lo que promete `AGENTS.md` y lo que el código aplica

- **Formulador:** aislamiento multi-tenant con defensa en profundidad genuina — RLS real y filtro `WHERE tenant_id = p_tenant_id` explícito en cada RPC `SECURITY INVOKER`. **Punto sin verificar con impacto real:** el `Authorization: Bearer` que llega a Supabase es siempre un ID token de Firebase (nunca un JWT de Supabase Auth) — esto solo hace que PostgREST reconozca el rol `authenticated` si el proyecto Supabase tiene configurado "Third-Party Auth" con Firebase, algo que se configura en el dashboard de Supabase y no es verificable desde este repositorio. El aislamiento confirmable hoy es el de la cláusula SQL explícita en las RPC, no necesariamente el de la política RLS por rol. **Recomendado: prueba cruzada de dos tenants reales antes de tratar el aislamiento como comprobado end-to-end.**
- **Radar:** no tiene noción de tenant en absoluto — `radarData` es un array global compartido, y la cache key de `m1Pipeline.js` tampoco incluye tenant/uid. No es una fuga de datos hoy (las convocatorias son públicas), pero confirma que el axioma de aislamiento de `AGENTS.md` simplemente no aplica a este módulo porque no existe el concepto ahí.

3.7 Firestore — perímetro positivo, confirmado

`firestore.rules`: `audit_logs` legible solo por el email del dueño, escritura siempre denegada, catch-all deny-by-default. Sigue siendo el único perímetro "admin" verificable de todo el sistema.

4. Recomendaciones de seguridad, clasificadas por tensión con economía

Categoría A — Gratis o casi gratis, sin tensión con economía (hacer ya)

1. Rellenar `UPSTASH_REDIS_REST_URL / TOKEN` (local + Render) — sirve a economía Y seguridad a la vez. Costo: \$0.
2. Corregir el log falso de `FirebaseAdmin.js:22`.
3. Normalizar `query ( trim()+toLowerCase() )` antes de hashear el cache key en `m1Pipeline.js`.
4. Rate-limit mínimo por `uid` (in-memory, mismo patrón barato que `activeQueries`) en `/api/radar/search` y `/api/chat`.
5. Corregir el comentario engañoso `server.js:279` ("acceso público").
6. Validación mínima de esquema en `POST /api/settings/trello`.

Categoría B — Mejora de seguridad real, costo no trivial (decisión del usuario)

1. **Arreglar el login del SPA (\$0)** — prerequisite de todo lo demás. Costo: tiempo de ingeniería, no dinero recurrente. Es la pieza de trabajo más grande de este informe.
2. **Verificar/corregir la cadena Firebase↔Supabase para RLS (\$3.6)** — costo: tiempo de investigación/configuración.
3. **Rate limiting a nivel de infraestructura** (Cloudflare/WAF delante de Render) — puede implicar costo recurrente. **No hacer todavía** dado el tráfico real actual.

Categoría C — Optimización de economía que podría degradar seguridad si se hace mal (advertencia)

1. Cachear agresivamente resultados de Radar amortiza costo entre usuarios — **advertencia:** si Radar alguna vez incorpora datos por-tenant (favoritos, notas privadas), la cache key debe incluir tenant/uid desde el primer commit de esa función, no después.
2. `SERVICE_KEY` como fallback en `supabaseClient.js` bypasea RLS por completo — correcto donde ya se usa (infraestructura interna), no expandirlo a nuevas rutas de usuario final sin revisión deliberada.

5. "SaaS de alto nivel" — brecha frente al estado actual

Hoy el sistema es mono-tenant de facto (una sola cuenta admin, que ni siquiera puede loguearse desde la UI real hoy, \$0).

- **Formulador:** tiene resuelta la parte más difícil (RLS + RPC atómicas), pero `tenant_id` se deriva 1:1 de un `uid` de Firebase — cada usuario ES su propio tenant. No hay tabla de organizaciones/equipos, ni roles internos, ni invitaciones. Es aislamiento multi-usuario correcto, no multi-tenant-con-equipos todavía.
- **Radar:** no tiene noción de tenant en absoluto. Si el producto necesita radar personalizado por tenant, es diseño de dominio nuevo desde cero.
- **RBAC admin:** tiene ahora un camino de asignación real (custom claim), pero el eslabón que lo activaría en un flujo de usuario real no existe.
- **Secuencia recomendada, sin fabricar la decisión de producto:** (1) arreglar login real → (2) decidir si "tenant = usuario individual" alcanza para el horizonte actual o hace falta "tenant = organización con miembros" → (3) si hace falta lo segundo, es un módulo de dominio nuevo, no un parche sobre el controlador actual.

6. Techo real de la arquitectura actual (escalabilidad)

Orden de qué se rompe primero al crecer tráfico, con un solo proceso Render free sin réplicas:

- 1. **Ya se rompe hoy, sin más tráfico:** el ciclo de suspensión/reactivación del tier free de Render resetea `radarData` y (mientras Upstash siga vacío) las sesiones, sin mediar deploy.
- 2. **WebSocket single-process:** sin adaptador pub/sub entre instancias — si algún día hay réplicas, un cliente en la instancia A nunca vería un broadcast disparado en la instancia B. No resuelto por los cambios de la sesión previa (la persistencia de sesiones no toca el WS).
- 3. **activeQueries por-proceso:** correcto hoy (diseño deliberado), deja de proteger nada útil con más de una instancia.
- 4. **m1Pipeline.js** : sin bottleneck de diseño propio — el límite real sería de cuota de Anthropic/Tavily a nivel de cuenta, no del código.
- 5. **Tiers gratuitos de terceros:** margen amplio hoy en Firebase/Firestore; Supabase con riesgo de pausa por inactividad ya mitigado; Upstash (una vez activado) tiene límite diario de comandos, lejos hoy.

Qué ya quedó bien encaminado: sacar `activeSessions` del `Map` local es, literalmente, el primer paso obligatorio para escalar a múltiples instancias — sin él, cada instancia habría tenido usuarios "logueados" distintos. Con Upstash realmente activado (§2/§4-A1), esa pieza queda resuelta para sesiones. Falta el WS (punto 2) y el anti-flood (punto 3) para que **todo** el sistema soporte múltiples instancias.

Qué es barato preparar ahora sin activar infraestructura nueva: diseñar la cache key de Radar pensando en tenant desde ya; si se construye el rate-limit por `uid`, hacerlo detrás de una función intercambiable (`checkQuota(uid)`) que pueda migrar a Redis sin reescritura.

Qué genuinamente debe esperar: multi-región, Kubernetes, adaptador Redis pub/sub para WS, o el módulo de organizaciones/equipos — nada de esto está justificado por el tráfico real actual (§0). Construirlo ahora sería exactamente el gasto de ingeniería prematuro que el criterio de economía pide evitar.

7. Deuda arquitectónica estructural (no directamente de costo)

Hallazgo	Recomendación	¿Ahora o esperar?
<code>server.js</code> monolito de wiring (7+ routers montados directo, sin DI)	Extraer a módulo de composición	Esperar — no bloquea nada hoy
<code>formulador / radar</code> sin capa de dominio propia	Ya documentado en la Radiografía Técnica, sigue vigente	Esperar — coincide con el resto del árbol
<code>pgClient.js</code> — 100% muerto, <code>DATABASE_URL</code> vacío	Eliminar	Hacer ya — cero riesgo
<code>backend_fastapi.py / server-sim.js</code> — sin camino a producción; FastAPI con CORS abierto (<code>allow_origins= ["*"]</code>)	Decisión del usuario: mantener documentados como no-producción, o eliminar igual que se hizo con los agentes Python rotos de <code>Proy_03</code>	Decisión pendiente
Comentario engañoso <code>server.js:279</code>	Corregir	Hacer ya , trivial
Log falso <code>FirebaseAdmin.js:22</code>	Corregir	Hacer ya , trivial
Vestigio <code>DEV_HEADERS / X-Local-Dev</code> en CORS	Eliminar	Hacer ya , trivial

8. Tabla de prioridad final unificada (economía + seguridad + escalabilidad + deuda estructural)

#	Prioridad	Acción	Ejes que resuelve	Costo
1	Alta	Rellenar credenciales Upstash (<code>.env</code> local + Render)	Costo + Seguridad + Escalabilidad	\$0, minutos
2	Alta	Arreglar login real del SPA (Firebase Auth client + wiring)	Seguridad + prerequisite de negocio	Tiempo de ingeniería, ítem más grande de la lista
3	Alta	Prueba cruzada de 2 tenants reales: ¿RLS por rol <code>authenticated</code> aplica de verdad vía Firebase→Supabase?	Seguridad + SaaS de alto nivel	Tiempo de investigación/configuración
4	Alta	Rate-limit mínimo por <code>uid</code> en <code>/api/radar/*</code> y <code>/api/chat</code>	Costo + Seguridad	Bajo, horas
5	Media	Normalizar query antes de hashear cache key	Costo	Minutos
6	Media	Diseñar (sin activar aún) cadencia real de conexión WS↔ <code>m1Pipeline.js</code> : on-demand o cron de baja frecuencia con cache compartido	Costo + Escalabilidad	Bajo-medio
7	Media	Eliminar <code>pgClient.js</code>	Deuda estructural	Trivial
8	Media	Corregir comentario <code>server.js:279</code> , log <code>FirebaseAdmin.js:22</code> , eliminar <code>DEV_HEADERS</code>	Deuda + Seguridad	Trivial
9	Media	Validación mínima en <code>POST /api/settings/trello</code>	Seguridad	Bajo
10	Baja	Decidir "tenant = usuario" vs. "tenant = organización"	SaaS de alto nivel	Alto si se construye — esperar decisión de producto
11	Baja	Pub/sub para WS entre instancias	Escalabilidad	Medio-alto — esperar tráfico real
12	Baja	Rate limiting de infraestructura (WAF/Cloudflare)	Seguridad	Puede costar dinero recurrente — esperar
13	Baja	Refactor <code>server.js</code> /capa de dominio	Deuda estructural	Medio — esperar
14	Baja	Decidir destino de <code>backend_fastapi.py</code> / <code>server-sim.js</code>	Deuda + Seguridad marginal	Bajo si se elimina

Nota metodológica

Este es un entregable de auditoría y recomendación — de solo lectura, sin cambios aplicados en esta ronda, sin aprobar ni bloquear ninguna acción. Cualquier ítem de la tabla §8, al proponerse como cambio concreto de código, debe pasar por el gate de arquitectura habitual (`.claude/agents/architect.md` o `node agents/000_Orquestador.cjs --aprobar-diseno`) antes de implementarse, tal como el resto del ecosistema.

Rutas citadas: `server.js` , `src/shared/infrastructure/FirebaseAuthMiddleware.js` , `session-manager.js` , `cache.js` , `FirebaseAdmin.js` , `AuditLogger.js` , `src/modules/radar/m1Pipeline.js` , `src/modules/formulador/FormuladorPgController.js` , `supabaseClient.js` , `pgClient.js` , `migraciones 001/004/005` , `firestore.rules` , `public/src/pages/InicioPage.jsx` , `public/src/main.jsx` , `public/test_auth.html` , `public/src/RadarApp.jsx` , `public/src/App.jsx` , `scripts/generar_reporte.cjs` , `scripts/db-check.js` , `render.yaml` , `AGENTS.md` , `.env` , `.gitignore` .